

Overview

Eyes-Only Web Navigation is a browser extension that allows users to control the mouse cursor using **eye gaze** and perform clicks using **blinks**. The goal is to enable web navigation for users with **severe motor impairments** such as ALS, quadriplegia, or temporary injuries that prevent mouse use.

The system uses **webcam-based eye tracking** to estimate the user's gaze location and move the mouse cursor accordingly. Blink detection triggers a simulated mouse click on the current active tab.

The prototype focuses on the core interaction:

- **Eye movement** → **cursor movement**
- **Blink** → **click**

This allows users to interact with any website without using their hands.

Goals

Primary Goals

- Allow users to control the mouse cursor using eye gaze.
- Detect intentional blinks and convert them to mouse clicks.
- Work on **any website in the active browser tab**.
- Run using only a **standard webcam** (no specialized hardware).

User Flow

Step 1 — Enable Eyes-Only Mode

User activates the extension.

The extension:

- requests webcam permission
- initializes eye tracking
- overlays cursor control on the active tab.

Step 2 — Calibration (optional but recommended)

User looks at several points on the screen.

Example calibration points:

- top left
- top right
- center
- bottom left
- bottom right

This improves gaze mapping accuracy.

Step 3 — Cursor Movement

User gaze position is estimated from webcam input.

The system maps gaze coordinates to screen coordinates: gaze position → cursor position

Cursor updates continuously.

Step 4 — Blink Detection

The system detects eye blinks using facial landmark tracking.

When a blink is detected: blink → simulate mouse click

The click is triggered on the current cursor location.

System Architecture

High-level architecture:

Webcam Input



Face Landmark Detection



Eye Tracking Model



Gaze Coordinate Estimation



Cursor Position Mapping



Browser Cursor Movement



Blink Detection → Mouse Click

Components

1. Webcam Capture

Captures video frames from the user's webcam.

Implementation: `navigator.mediaDevices.getUserMedia()`

Frames are passed to the eye tracking model.

2. Face Landmark Detection

Detects facial landmarks including eye positions.

Possible libraries:

- MediaPipe Face Mesh
- WebGazer.js
- TensorFlow.js models

The landmarks help compute:

- pupil location
- eye aspect ratio (for blink detection).

3. Gaze Estimation

Estimates where on the screen the user is looking.

Approach:

1. Track eye region landmarks
2. Estimate gaze vector
3. Map gaze to screen coordinates

Output: (x, y) gaze location on screen

4. Cursor Controller

Moves the cursor based on gaze position.

Implementation options:

Overlay Cursor

Render a visual cursor inside the webpage.

Native Cursor Control

Simulate pointer movement using DOM events.

For the prototype we use an **overlay cursor** because browsers restrict direct cursor control.

5. Blink Detection

Blink detection uses **Eye Aspect Ratio (EAR)**.

EAR formula (simplified):

$EAR = (\text{vertical eye distance}) / (\text{horizontal eye distance})$

When EAR falls below a threshold: blink detected

Example thresholds: $EAR < 0.20 \rightarrow \text{blink}$

To prevent accidental clicks: blink must last > 100 ms

6. Click Simulation

When blink detected: dispatch mouse click event

Example event: `element.dispatchEvent(new MouseEvent("click"))`

The click occurs on the element under the cursor.

Extension Architecture

The extension contains three main components.

Background Script

- manages extension lifecycle
- handles permissions

Content Script

Runs inside the active tab.

Responsibilities:

- cursor rendering
- DOM click simulation
- communication with tracking system.

Eye Tracking Module

Runs in the extension UI or injected page.

Responsibilities:

- webcam processing
- gaze estimation
- blink detection

Outputs:

cursor_position
blink_event

Data Flow

Webcam Frame



Face Detection



Eye Landmark Detection



Gaze Position



Cursor Update



Blink Detection



Click Event

Performance Considerations

To ensure smooth interaction:

Target processing speed: 15–30 FPS

Optimizations:

- downscale webcam frames
- process only eye regions
- debounce blink detection

Safety and Error Handling

Potential issues:

False Blink Detection

Mitigation:

- minimum blink duration
- cooldown period between clicks

Example: click cooldown = 500 ms

Poor Lighting

System may lose face tracking.

Mitigation:

Display warning:

Face not detected

Cursor Jitter

Small gaze movements may cause cursor jitter.

Mitigation:

Apply smoothing filter:

`smoothed_position = weighted_average(previous_positions)`

Accessibility Impact

This system enables web interaction for users who:

- cannot use a mouse
- cannot use a keyboard
- have limited motor control

Potential users include:

- ALS patients
 - quadriplegic users
 - severe arthritis
 - temporary injuries.
-